NoWave

DOCUMENTATION TECHNIQUE

29 OCTOBRE 2026

Preuves et résultats des tests

Application maritime NoWave

Bilan des tests locaux et des contrôles CI

2 563	4	297
tests réussis	tests ignorés	exécution CI

1. Objectif

Ce document présente la manière dont nous avons vérifié les principales parties de NoWave et les résultats disponibles. NoWave est une application de signalements maritimes participatifs : elle permet notamment de consulter une carte, de publier des observations et de recevoir des alertes de proximité. Le projet comprend aussi un site public et une interface d'administration.

Les tests nous aident à détecter des erreurs pendant le développement et à vérifier qu'une modification ne casse pas un comportement déjà prévu. Ils ne garantissent pas l'absence totale de défauts : leurs résultats dépendent des scénarios et des environnements utilisés.

2. Notre démarche

Nous avons regroupé les vérifications par partie du projet, puis par type de test. Les tests automatisés sont exécutés par des outils ; ils peuvent être relancés après une modification. Les essais sur appareil permettent de compléter ces résultats pour des fonctions liées au téléphone, comme la caméra ou les capteurs.

- **Tests unitaires :** vérifient une règle ou une fonction isolée.
- **Tests d'interface :** vérifient le comportement d'un écran ou de ses éléments.
- **Tests d'intégration et de base de données :** vérifient la collaboration entre plusieurs parties, notamment les échanges avec la base de données.
- **Tests dans un navigateur :** reproduisent des actions dans une interface web.
- **Essais sur appareil et mesures :** observent certains comportements sur un téléphone réel, dans des conditions précisées.

Les contrôles automatiques du projet sont aussi lancés par GitHub Actions, lors de certaines modifications. Les rapports produits servent de preuves : ils sont associés à une exécution et à une version du code. Leur présence ne signifie pas que toutes les fonctions ont été essayées dans des conditions réelles.

3. Périmètre des tests du projet

L'inventaire ci-dessous couvre les tests propres à NoWave présents dans la version `main` identifiée au chapitre suivant. Les tests internes aux bibliothèques téléchargées ne sont pas comptés comme des tests de l'équipe.

Partie	Tests et emplacement	Résultat du 29 octobre
Serveur : API	Comptes, appareils, signalements, photos, alertes, carte et administration - `backend/tests/api/`	441 réussis
Serveur : règles	Règles métier, notifications, données maritimes, authentification et stockage - `backend/tests/unit/`	792 réussis, 2 ignorés
Base de données	Schéma, données, positions, alertes, modération et opérations simultanées - `backend/tests/database/`	289 réussis
Administration	Navigation, carte, style et surveillance - `backend/tests/web/`	30 réussis
Administration	Parcours dans un navigateur avec une base de test - `backend/tests/browser/admin.spec.cjs`	28 réussis
Carte d'administration	Carte Mapbox réelle dans un navigateur - `backend/tests/browser/mapbox.spec.cjs`	2 ignorés

Partie	Tests et emplacement	Résultat du 29 octobre
Mobile : fonctionnalités	Connexion, carte, signalements, alertes, caméra et profil - `mobile/test/features/`	Inclus dans les 950 réussis
Mobile : fonctions communes	API, appareil, localisation, carte, notifications, capteurs et texte - `mobile/test/core/`	Inclus dans les 950 réussis
Mobile : autres tests	Intégration, performance et tests généraux - `mobile/test/`	Inclus dans les 950 réussis
Site public	Données, observations, carte, parcours, gestes et configuration - `web/src/*.test.js`	33 réussis

Le dossier `backend/tests/integration/` existe, mais ne contient pas de fichier de test dans cette version. Les vérifications qui associent réellement plusieurs composants se trouvent notamment dans les suites de base de données et de navigateur. La suite Mapbox demande une configuration et un accès au service ; sans configuration, le scénario est ignoré dans les deux formats testés, ordinateur et mobile. La CI actuelle lance `admin.spec.cjs`, mais pas `mapbox.spec.cjs`. Les essais sur appareil et les contrôles de construction et de déploiement sont présentés séparément plus loin.

4. Résultats du 29 octobre 2026

Le 29 octobre 2026, nous avons relancé les suites ci-dessous sur une copie isolée de `main`, au commit `425c81d25f8cefdba6bfbb6122e146f27493c6b8`. Les résultats concernent cette version et cette exécution locale.

Partie et groupe	Résultat du 29 octobre
Application mobile Flutter	950 réussis
Serveur : API	441 réussis
Serveur : tests unitaires	792 réussis, 2 ignorés
Base de données	289 réussis
Site web public	33 réussis
Administration JavaScript	30 réussis
Parcours navigateur d'administration	28 réussis
Carte Mapbox réelle dans le navigateur	2 ignorés, configuration absente
Total des suites ci-dessus	**2 563 réussis, 4 ignorés**

Les deux tests serveur ignorés nécessitaient respectivement des archives maritimes locales absentes et une configuration Nginx spécifique non activée. La base de données a été créée dans un conteneur de test isolé. Les tests locaux ont utilisé Flutter 3.47.4 et Node 26.9.0, tandis que la CI prévoit Flutter 3.47.5 et Node 24. Pour Python,

un environnement local complété a été utilisé : certaines versions de dépendances diffèrent du fichier verrouillé, car son installation complète n'a pas abouti. Les résultats locaux restent donc à compléter par ceux de la CI.

CI GitHub du 29 octobre : l'exécution n° 297 de la PR 139 a réussi. Elle a testé la fusion préparée par GitHub (`b9ca412`), entre `dev` (`ad54ffd`) et la version de `main` alors en place, avant la fusion finale `425c81d`. Il s'agit d'une exécution distincte des tests locaux ci-dessus.

Les contrôles Python, Flutter, base de données et parcours d'administration, site web, interface d'administration, ainsi que les audits des dépendances et des images ont réussi. Le déploiement de production a été **ignoré** dans cette exécution : elle ne constitue donc pas une preuve de déploiement. Les archives de résultats ont été conservées avec les preuves locales.

5. Application mobile

Les tests de l'application couvrent plusieurs domaines du projet : la carte et ses commandes, les signalements et leurs fiches, les alertes, le profil, la localisation et la caméra. Ils comprennent des tests de règles et de données ainsi que des tests d'écrans et de leurs interactions.

Exemples de vérifications : affichage et navigation sur la carte, fonctionnement du formulaire de signalement, accès aux informations de profil, traitement des positions et des alertes. La suite comprend également des tests d'intégration et des mesures de performance.

Preuve et résultat : la relance locale du 29 octobre indique 950 tests Flutter réussis pour la version précisée au chapitre 4. Les fichiers de tests sont regroupés dans `mobile/test/` et les résultats automatiques Flutter sont conservés parmi les artefacts de la CI.

Limite : ces tests ne remplacent pas une recette complète sur différents téléphones. En particulier, les notifications réelles, les capteurs et les conditions de connexion doivent être vérifiés sur appareil.

Tests sur mobile et suivi dans Linear

En complément des tests automatisés, les tests réalisés sur mobile ont été consignés dans notre espace Linear, directement dans les issues qui nécessitaient une vérification.

Ces essais complètent les tests automatisés pour les fonctionnalités liées à l'appareil.

6. Serveur et base de données

Le serveur reçoit les demandes de l'application et du site, applique les règles du projet et échange avec la base de données. Les tests vérifient notamment les demandes liées aux utilisateurs, aux signalements, aux alertes et aux données cartographiques. Les tests de base de données contrôlent également son fonctionnement et ses échanges avec le serveur.

Preuve et résultat : la relance du 29 octobre a donné 1 233 tests API et unitaires réussis, 2 ignorés et 289 tests de base de données réussis. Les journaux et rapports détaillés sont conservés dans le dossier de preuves sur le Bureau.

Ce qui est vérifié : les tests unitaires examinent les règles de création de comptes et de signalements, les alertes, les positions et les données maritimes. Les tests API vérifient les réponses du serveur et les cas d'erreur. Les tests de base de données vérifient notamment la conservation et la lecture des données, les migrations et certaines opérations simultanées.

Limite : certains services peuvent être remplacés par des simulations pendant les tests. Un résultat positif dans cet environnement ne démontre donc pas à lui seul la disponibilité ou le fonctionnement de tous les services externes réels.

7. Site web et administration

La relance locale du 29 octobre a donné 33 tests réussis pour le site public et 30 pour les tests JavaScript de l'administration. L'administration est également vérifiée par des tests du serveur, des tests JavaScript et des parcours dans un navigateur. Ces derniers permettent de reproduire des actions sur l'interface avec une base de données de test.

Parcours navigateur : 28 parcours ont réussi sur ordinateur et en format mobile ; le scénario avec la vraie carte Mapbox a été ignoré dans les deux formats faute de configuration. Les parcours utilisent une base de test ; l'identité, le stockage des photos et certains services sont simulés. La suite utilisant la vraie carte Mapbox nécessite une configuration distincte.

8. Contrôles complémentaires

La CI vérifie également la qualité et la mise en forme du code, la construction du site et le démarrage du service web. Elle effectue aussi des contrôles liés aux dépendances et aux images de production. Ces contrôles complètent les tests fonctionnels : ils ne doivent pas être ajoutés au nombre de tests réussis. La CI citée au chapitre 4 vérifie la fusion proposée avant son intégration ; le déploiement n'y a pas été exécuté.

11. Sources et preuves

Les fichiers ci-dessous sont les journaux de l'exécution locale du 29 octobre. Ils accompagnent ce document dans le dossier `NoWave-preuves-tests`.

- Application mobile `(NoWave-preuves-tests/nowave-main-flutter.log)` : résultat de la suite Flutter.
- Serveur API et tests unitaires `(NoWave-preuves-tests/nowave-main-python.log)` et rapport détaillé `(NoWave-preuves-tests/nowave-main-python.xml)` : résultats et tests ignorés.
- Base de données `(NoWave-preuves-tests/nowave-main-database.log)` et rapport détaillé `(NoWave-preuves-tests/nowave-main-database.xml)`.
- Site web `(NoWave-preuves-tests/nowave-main-web.log)` et administration JavaScript `(NoWave-preuves-tests/nowave-main-admin.log)`.
- Parcours navigateur `(NoWave-preuves-tests/nowave-main-browser.log)` et rapport détaillé `(NoWave-preuves-tests/results.json)`.

La configuration des contrôles automatiques se trouve dans `.github/workflows/checks.yml` du dépôt. Les limites des parcours navigateur sont décrites dans `backend/tests/browser/README.md`. Le fichier `nowave-main-install.log` du dossier de preuves concerne une tentative de préparation des dépendances ; ce n'est pas un résultat de test.

Archives de la CI GitHub

Les archives ci-dessous proviennent de l'exécution n° 297. Elles sont conservées localement car les artefacts GitHub ont une durée de conservation limitée. Leurs empreintes ont été comparées à celles publiées par GitHub.

- Résultats Python `(NoWave-preuves-tests/CI-python.zip)` et base de données `(NoWave-preuves-tests/CI-database.zip)`.
- Résultats Flutter et couverture `(NoWave-preuves-tests/CI-flutter.zip)`.
- Résultats des parcours d'administration `(NoWave-preuves-tests/CI-admin-browser.zip)`.
- Résultats du site web `(NoWave-preuves-tests/CI-web.zip)`.
- Audits des images de production `(NoWave-preuves-tests/CI-audit.zip)`.